Goals

Goals: model can predict new edges in a graph over time

- This will likely involve merging a GCN (likely attention based) with a recurrent model, likely an LSTM

Notes

Notes: Model will be created with global node pool. Some nodes will be unconnected. This is ok.

Representation

First need to consider how to represent the text in edges and nodes

Edge embeddings

Edge embeddings

- need to be a limited subset that
- embeddings will be learned by the model
- Ex: born, died, friends, enemies, killed_by, etc

Node text

Then how to represent node text:
2 options

- One-hot encoding (in hindsight I dont think this is plausible. Too little info)
  - Pros
    - Easy decode rule (take softmax and sample probabilities)
    - Super easy to implement (nn.Embedding layer)
  - Cons
    - Less representative and would probably worsen the model
    - Can get big fast because of the copious amount of proper nouns in the dataset
- Actual embedding layer
  - Generate separate embeddings for different characteristics
    - name

        * description
        * class
    - embeddings will be merged in an embedding layer in the model
    - Pros
        * Much more representative
        * Size is more manageable
    - Cons
        * Much more difficult to implement
            · imported models will probably struggle with this task because of all the proper nouns, but adding our own embedding layers will make the model more expensive and difficult
            · create node embeddings with description integrated for better preserved meaning

The best idea is to go with multiple pretrained embeddings of names, class, and description. Then put an embedding head that takes in each embedding vector, synthesizes it, and projects down into GNN dim

- dataset will be a concatenation of the three because we can just leave it as the concat or project down

Text embedding could also be merged with structural embedding in some way, but the structure will probably be learned via convolutions so its probably ok.

With embedding taken care of each node will be represented by an embedding and we have the starting structure for the graph.

Embedding layer

Embedding layer:

- simple MLP that takes all the embedding vectors representing a model and projects them into hidden dim

GNN Update

Update Rule:

$$h_i^{(t+1)} = \text{Non-Linearity}\left(\sum_j a_{ij} \cdot m_{ij}\right)$$

where a_{ij} is an attention score and m is message (linear combination of messages)

- attention scores are calculated by

$$a_{ij} = \text{Non-Linearity}(a(z_i, z_j, \text{edge}_{ij}))$$

  where a is learned attention function and

$$z = W_n \cdot h$$

  Do it only over neighbors

  - gotta do softmax to get a_{ij}

- messages (need a way to synthesize edges and nodes)

$$m_{ij} = W_h \cdot h_j^{(t)} + W_r \cdot \text{edge}_{ij}$$

Multi-head attention

- Multi head attentions should be considered after single head is finished. Basically the same thing but project down and concat

- dim has to be divisible by number of heads
- Same set of the above equations for each head
- at end concat output of each head for full
- for efficiency can use the same weights for all heads. Its not as powerful as full multi-head attention, but it still captures different relationships

Further considerations

Further considerations

- since we are using sum should probably use layer norms to keep things from exploding and maybe residuals
- Something like this

$$h^{(t+1)} = \text{GNNLayer}(h^{(t)})$$
$$h^{(t+1)} = \text{LayerNorm}(h^{(t+1)} + h^{(t)})$$

- can also add a feedforward network after attention like GNNs

$$h^{(t+1)} = \text{LayerNorm}(h^{(t+1)} + \text{MLP}(h^{(t+1)}))$$

- dropouts could also be useful for better generalization

Running this for a bit will give us rich contextual embeddings for each node, which we can then use for link prediction and node generation

LSTM Strategies

First update LSTM hidden strategies

- Two ways to do this.
  - 1: Full graph embeddings
    - $x = \text{ReadOut}(H)$
    - Standard LSTM update using x
    - Problem with this is that it captures global patterns but loses fine-grained info from singular nodes
  - 2: Store a hidden state for each node
    - normal LSTM update conditioned on the node's hidden state
    - Stores much more fine-grained info, at the cost of not having as strong a representation for the graph as a whole

Link prediction

Link prediction

- strategies
  - 1: predict only changes
  - 2: predict entire graph
- Split into two parts
- 1: Candidate proposal
- 2: Candidate scoring

Loss

Loss:

- Binary cross entropy with the next graph

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(p_i) + (1 - y_i)\log(1 - p_i)\right]$$

$$p_i = \sigma(\text{MLP}(i, j, \text{edge}, i_{\text{hidden}}, j_{\text{hidden}}))$$

- existing edges are labeled one and ones that dont exist are 0
- this will hopefully capture loss for both negative and positive samples

Generation

Generation:

- Autoregressive generation
- Start with one graph, run the network over it to get the next, then pass that graph back into the model and so on

Data considerations

Data considerations:

- structure has to be suitable for pytorch
  - this is more of a code thing. We're gonna have to figure this out
- a strict typing system would provide a lot of structure that could be exploited
  - unless the data already has it we would probably need a llm pipeline to type everything
  - built into wikidata?
- constrained number of relationships
  - this could also be a pain if not in dataset